

Characterizing and Bridging the Diagnostic Gap in eBPF Verifier Rejections

Anonymous Author(s)

Abstract

eBPF lets developers run custom programs inside the Linux kernel, where a static verifier must prove each one safe before it loads. When the verifier rejects a program, it returns a log of bytecode instructions and register states, and the error flags the operation it could not prove safe. While the log records every step the verifier took, it is usually hard for a developer to tell why the program was rejected or what to change. To identify the gap, we build BPFIX-BENCH, a corpus of 235 rejections reproduced from real projects, and conduct an empirical study. We find that the log is hard to use because it does not show two things a repair needs: the root cause and the responsible layer. To address these limitations, we present BPFix, a userspace tool that recovers both from the log alone in a single diagnostic. We feed the BPFix diagnostic to a downstream repair model and test whether the model repairs the program. Across three models on the 75-case BPFix-TEST repair suite, BPFix raises one-shot repair success by 10.7–21.4 percentage points, from 0.0–37.3% with the raw log to 10.7–50.7% with the BPFix diagnostic.

CCS Concepts: • **Software and its engineering** → *Formal software verification*; **Development frameworks and environments**; *Compilers*.

ACM Reference Format:

Anonymous Author(s). 2026. Characterizing and Bridging the Diagnostic Gap in eBPF Verifier Rejections. In *Proceedings of ACM SIGCOMM Workshop on eBPF and Kernel Extensions (eBPF '27)*. ACM, New York, NY, USA, 8 pages.

1 Introduction

eBPF now runs production networking, security, and observability inside the Linux kernel [7]. Every program must first pass a static safety verifier, which rejects any program it cannot prove safe [8]. When the verifier rejects a program, it returns a log of bytecode instructions and register states, ending in an error message. While the log records every step the verifier took, it is usually hard for a developer to tell why the program was rejected or what to change. As a result, the developer resorts to time-consuming trial and error to satisfy the verifier [5].

To identify the gap, we construct BPFIX-BENCH, a corpus of 235 rejections reproduced from real projects. We read each rejection's source and fix to assign the responsible layer: 191 are source bugs, and the other 44 are rejections of correct code, caused by the compiler, the environment, or the verifier. For the source bugs, we further classify the root cause into

12 categories. We find the verifier's log difficult to use for repair, for two reasons. First, it does not reveal the root cause: a single error message can arise from as many as seven root-cause categories. Second, it carries no field for the responsible layer. Tools that help developers read the verifier's output, such as PrettyVerifier [20] and the BPF Verifier Visualizer [12], enhance or visualize it but do not reconstruct the verifier's reasoning.

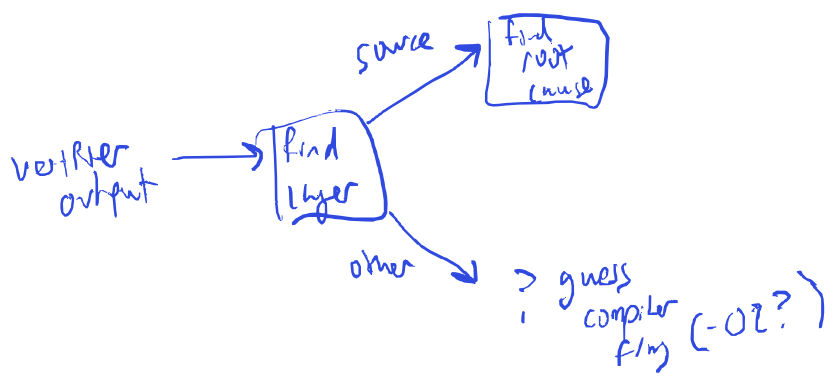
While the verifier's log lacks the two categories of information described above, a tool can recover them from the log to help diagnose and fix the program. The verifier builds a safety proof as it checks the program instruction by instruction, then discards the proof at the rejected operation and reports only the symptom. Under `log_level=2`, though, it records its state after every instruction [16, 24]. From these states, a tool reconstructs the discarded proof, which reveals the missing information.

To help a developer diagnose and fix a rejection, we introduce BPFix, a userspace tool that turns the verifier's rejection log into a compiler-style diagnostic. Figure 1 shows BPFix on a real rejection, where the verifier returns a single register-level line. From the same log, BPFix maps the terminal message to the safety property the verifier could not prove. BPFix then traces the property through the per-instruction states, recording where it is established and where it is lost. From the reconstructed proof, BPFix renders a diagnostic that gives the root cause and the responsible layer.

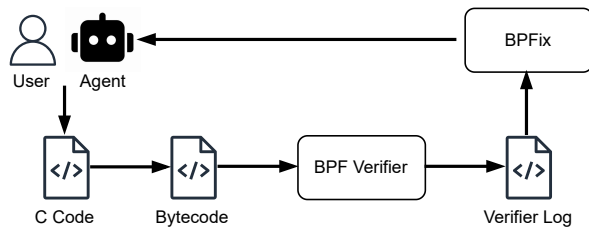
We test whether the BPFix diagnostic enables more repairs than the raw verifier log. We give a language model the buggy program with either the raw log or the BPFix diagnostic, on BPFix-TEST, a suite of 75 repair tasks, each checked by an executable oracle independent of BPFix (§4). Replacing the raw log with the diagnostic raises one-shot repair under all three models we test. On Qwen3.6 27B, the model repairs 22 of the 75 cases from the raw log and 38 from the diagnostic.

This paper makes the following contributions:

- We conduct an empirical study on BPFIX-BENCH to identify the gap between the verifier's log and a repair.
- We build BPFix, a tool that recovers the root cause and responsible layer from the verifier's discarded proof.
- We build two datasets: BPFIX-BENCH, a corpus of 235 reproduced rejections for the empirical study, and BPFix-TEST, 75 repair tasks with executable oracles for evaluating repair.



doesn't always seem 2 steps



C Code

```

263 if (ipv4_hdr)
264     udph = (void *)ipv4_hdr + sizeof(*ipv4_hdr);
265 else
266     udph = (void *)ipv6_hdr + sizeof(*ipv6_hdr);
267 if (udph + sizeof(struct udphdr) > data_end) // bounds-checked
268     return 1;
269
270 dst_port = __constant_ntohs(((struct udphdr *)udph)->dest); // rejected

```

Verifier Log

```

from 31 to 34: ... R5_w=40 ...
; if (udph + sizeof(struct udphdr) > data_end) @ prog.c:267
35: (07) r3 += 8 ; R3=48
36: (2d) if r3 > r2 goto pc+4 ; R2=pkt_end() R3=48
; dst_port = ...((struct udphdr *)udph)->dest; @ prog.c:270
37: (69) r2 = *(u16 *) (r5 + 2)
R5 invalid mem access 'scalar'

```

BPFix output

```

error[BPFIX-E006]: verifier-visible compiler lowering hides the required
proof
= class: lowering_artifact
= confidence: medium
= next action: provenance
--> prog.c:270
|
263 | if (ipv4_hdr)
| ----- nearby source context for pointer provenance
267 | if (udph + sizeof(struct udphdr) > data_end)
| ----- verifier state changes
|         from pkt to scalar before the rejected access
270 | dst_port = __constant_ntohs(((struct udphdr *)udph)->dest);
| ----- rejected
|         here: verifier sees a scalar where a pointer is required
|
= verifier[229]: R5 invalid mem access 'scalar'
= required proof: preserve a verifier-recognized pointer type at the
operation that requires a pointer
help: Reacquire a verifier-tracked pointer before the rejected dereference.
help: Use the packet pointer that received the data_end proof, or rederive
and recheck it before the load.

```

Figure 1. BPFix on a real rejection (Stack Overflow 53136145). The verifier returns one register-level line; from the same log, BPFix reconstructs the discarded proof to recover the root cause, responsible layer, and fix location.

2 An Empirical Study of Verifier Rejections

This section studies how a developer repairs a rejected program from the verifier's log, and what the log leaves out. We describe the verifier and its log (§2.1), build a labeled corpus of real rejections (§2.2), characterize the rejections in it (§2.3), and measure what their messages omit (§2.4).

2.1 Debugging Using the Verifier Log

Before the kernel runs an eBPF program, the verifier must prove it safe. The verifier checks a safety property at each operation, such as a memory access staying within a proven bound, exploring every path and tracking an abstract value for each register and stack slot. A program that fails a check

```

1 // correct source; -O0 lowers the field read into a scalar
2 static int add_one(int x) { return x + 1; }
3 int prog(struct xdp_md *ctx) { return add_one(ctx->rx_queue_index) & 1; }

verifier: R2 invalid mem access 'scalar'

```

Figure 2. A compiler-lowering rejection: correct source that the verifier reports as invalid mem access 'scalar'.

is rejected, and the verifier returns two artifacts: an error message naming the failed operation, and a trace of the abstract state after each instruction.

Repairing a rejection from this log takes two steps. The developer first determines which layer is responsible, because the fix need not lie in the program at all: the verifier checks compiled bytecode, so correct source is rejected whenever the compiler, the build, or the verifier is at fault. The program in Figure 2, for example, reads only a context field and is provably safe, yet the verifier rejects it with invalid mem access 'scalar' because -O0 lowers that read into a scalar dereference; the repair is to compile at -O2, and the source is never edited. Only once the source is at fault does the second step apply: identifying the safety requirement the program violated. The developer reads the error message and the trace together, following the failed register back through the recorded states to the property the verifier needed and the point at which the program lost it.

① find layer
② find source root cause (if layer is source)

2.2 BPFIX-BENCH: A Labeled Rejection Corpus

To study rejections that developers actually hit, we assemble BPFIX-BENCH, a corpus of real verifier rejections reproduced under one fixed toolchain. The corpus draws on 936 candidate reports from four sources: Stack Overflow questions, GitHub issues, GitHub fix commits, and kernel selftests. We rebuild each candidate and load it into kernel 6.15.11 with clang 18 at log level 2. Of the 936 candidates, 235 reject under this build and form BPFIX-BENCH. The rest are dropped because they do not reject under our toolchain, need a specific environment, or lack the source to rebuild.

Each BPFIX-BENCH case comes with the faulty source and the developer's own fix, both from its report. The source and fix give the ground truth for the bug and its repair. We use this ground truth to characterize the rejections in §2.3.

2.3 Characterizing the Rejections

We characterize each BPFIX-BENCH rejection by the two questions a repair answers: which layer is responsible, and, for a source bug, what the source got wrong.

Assigning the responsible layer. The source is at fault in 191 of the 235 rejections; these are genuine source bugs. In the other 44 the source is correct, and the fault lies in the compiler (18), the environment (14), or the verifier (12), where the repair is a build flag, a metadata or configuration change, or a verifier workaround, outside the program. The rest of this subsection examines the 191 source bugs.

any structured way to know which or give message, or we we focus on source bugs?

Table 1. The 191 developer bugs fall into 12 root-cause categories. A category is the specific source-level error the developer made; the *Cases* column gives how many bugs each accounts for.

Root-cause category	Cases
Unclamped scalar used as offset or length	24
Corrupted or stale dynptr object	23
Packet access without a bound on every path	22
Missing null check	19
Pointer type or provenance mismatch	16
Unverified address dereferenced	16
Index exceeds object capacity	15
Context or contract misuse	15
Unpaired resource reference	15
Interrupt flag not restored in order	11
Probe signature mismatched with the ABI	9
Oversized or uninitialized stack buffer	6
Total	191

Classifying the root cause. A source bug fails a safety property the verifier required, but that property does not name the source mistake. We label each source bug with both: the safety property the verifier could not prove, and the root cause in the source that left it unproven. The property is the verifier's requirement and the root cause the developer's mistake, so a source bug needs both. The 191 source bugs fall into 12 root causes (Table 1), most specific to eBPF, such as proving a packet bound or pairing a dynptr with its lifetime. Only two, missing null checks and out-of-bounds indexing, map onto a general taxonomy such as CWE, and together they cover 34 of the 191.

2.4 Measuring the Log's Diagnostic Gaps

The verifier reports a rejection in its own low-level terms: a terminal message carrying an *errno*, and, under a verbose mode, a trace of the abstract state at each instruction. The *errno* is too coarse to separate causes, as *EINVAL* alone accounts for 47% of the verifier's error returns. Even together, the message and the trace omit two things a repair needs: the responsible layer and the root cause.

The message names no layer. The message has no field for the responsible layer, and a single line can come from any layer. A source bug, a compiler-lowering rejection, and an environment issue all print the same invalid mem access 'scalar'; among the 28 rejections that carry this message, 24 are source bugs and four originate in the compiler (Figure 2). A developer who misreads the layer edits correct code while the repair lies in the build.

The message does not reveal the root cause. A single message maps to many root causes, so it cannot identify which one occurred. Grouping the 235 rejections by message template, after normalizing registers and offsets, reduces 167

Table 2. The four most frequent message templates, with the cases sharing each template and the distinct root-cause categories they span. A template normalizes registers and offsets to placeholders.

Terminal message template	Cases	Categories
R# invalid mem access 'scalar'	28	9
invalid access to packet	26	5
invalid access to map value	18	4
R# !read_ok	13	4

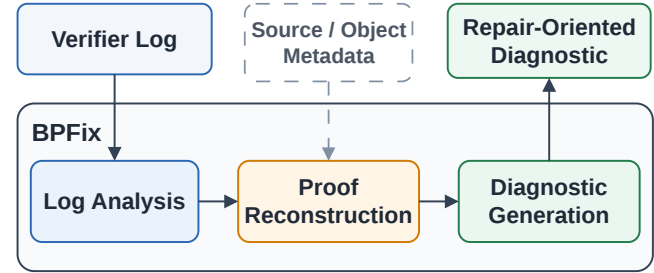


Figure 3. Overview of the BPFix workflow. BPFix analyzes verifier logs, reconstructs the missing proof, and generates a repair-oriented diagnostic.

distinct messages to 82 templates. Fifteen of these templates each span more than one root cause, and the most frequent spans 28 rejections across nine root causes (Table 2).

Across both gaps, the message leaves the responsible layer and the root cause for the developer to recover. To supply both from the log alone, we present BPFix (§3).

3 Design

A verifier rejection marks an operation at which the verifier cannot discharge a safety obligation. Its terminal message identifies this *rejection site*, but not necessarily where supporting evidence disappeared or where the source should be changed. From an existing verifier log, BPFix reconstructs the observable lifecycle of the required fact. We distinguish the rejection site, an observed *proof-break witness* after which logged evidence no longer supports that fact, and the *developer-chosen repair site*. A witness narrows the diagnosis, but is neither a minimal causal instruction set nor necessarily the location to edit.

Figure 3 summarizes the three-stage workflow: log analysis normalizes evidence around the rejection, proof reconstruction identifies and tracks the failed obligation, and diagnostic generation converts the resulting evidence into repair-oriented guidance.

3.1 Architecture and Data Flow

BPFix is a log-first pipeline whose only required input is an existing verifier log. Optional source or object metadata

how do you get mis?

repair

src vs. other?

how does he dev know mis? Isn't mis system supposed to help find it?

→ this is what you found, claim it

what happened to
"layer" in section 3?

improves instruction-to-source correlation but is not needed for log-level diagnosis.

The *Log Analysis* stage isolates the final rejection and extracts the terminal message, verifier-state and branch information, instruction identifiers, and available source annotations into a normalized evidence stream. The *Proof Reconstruction* stage maps the message to a proof family—such as pointer provenance, packet bounds, scalar range, non-nullness, stack initialization, or reference lifetime—instantiates the concrete fact required at the rejected operation, and tracks evidence for that fact. Its evidence record contains the required fact, rejection site, supporting evidence, observed establishment or loss events, and a proof-break witness when the log exposes one. The *Diagnostic Generation* stage selects primary and related spans, records their localization precision, derives a lifecycle-specific next action, and may assign a likely failure class and confidence using ordered evidence rules.

The result is a Rust-style plain-text diagnostic with a stable error identifier, the required fact, relevant spans and evidence, a witness when observable, and help: guidance. BPFix does not modify the kernel, rerun the verifier, or build an independent model of verifier semantics; missing metadata therefore affects source-level precision rather than instruction- or log-level diagnosis.

3.2 Reconstructing the Verifier-Visible Proof

BPFix first identifies the failed safety obligation. The terminal message anchors the rejected operation and immediate symptom, while nearby verifier state supplies parameters such as the affected register, access offset and size, scalar range, pointer type, or reference identifier. Together they instantiate the concrete fact required at the rejection site, without assuming that this site is the eventual repair location. For example, in Figure 1, R5 invalid mem access ‘scalar’ maps to a pointer-provenance obligation. From the rejected load and nearby state, BPFix infers that R5 must retain verifier-recognized packet-pointer provenance at BPF instruction 37.

Proof-family-specific extraction records three event types: a *proof-establishment event*, where logged evidence explicitly supports the fact; a *proof-loss event*, where a later logged state is incompatible with it; and the *rejection event*, where the verifier requires but cannot establish it. A missing snapshot or unprinted state is not itself proof loss. Each event retains its program location, relevant state, evidence source, and recoverable source span. When both a supporting and a later incompatible observation exist, their transition becomes the proof-break witness; otherwise, BPFix reports only the required fact and rejection site.

In the running example, an earlier state represents R5 as pkt (off=34, r=42), whereas a later state before instruction 37 represents it as scalar value 40. The final load dereferences R5, so BPFix reports the observed pointer-to-scalar

transition as the witness and instruction 37 as the rejection site. This witness marks only where provenance ceased to be visible in the printed evidence; it does not identify a unique causal instruction, account for unprinted paths, or determine the source line to edit.

3.3 Evidence-Aware Diagnostic Synthesis

Diagnostic synthesis adapts its guidance to the reconstructed lifecycle. If no establishment event is observed, BPFix recommends introducing or re-deriving the missing fact, for example through a same-path bounds check, null check, initialization, or reference acquisition. If the fact is visible and later lost, it recommends preserving or re-deriving it near the rejected use. If intermediate evidence is insufficient, it recommends conservative local re-establishment without claiming an unsupported witness.

The evidence record also determines localization. BPFix selects the most informative location as the primary span, reports relevant rejection, establishment, or loss locations as related spans, and states whether each is backed by source mapping, a BPF instruction identifier, or only a log position.

Responsibility attribution follows, rather than drives, proof reconstruction. Because the verifier observes bytecode after compiler lowering and loader-side relocation—including CO-RE updates to instruction immediate or offset fields at load time [15]—an observed transition alone cannot distinguish source, compiler, loader, or verifier causes. BPFix therefore applies ordered signals over terminal messages, lifecycle events, register states, helper context, and source annotations, preferring state-backed patterns to generic fallbacks. It reports a failure class and confidence only when supported; otherwise it preserves the evidence-backed diagnosis without forcing an attribution.

Developers use the resulting guidance to choose and validate a source-level repair through their normal compilation and verification workflow; BPFix neither rewrites the program nor treats the witness as the required edit location.

4 Evaluation

We evaluate BPFix with three questions:

- **RQ1.** Is the BPFix diagnostic a more actionable basis for repair than the raw verifier log?
- **RQ2.** Does the BPFix diagnostic supply the repair information the verifier message omits?
- **RQ3.** Can BPFix identify the responsible layer when several layers share one verifier message?

Datasets. We use two datasets. BPFix-TEST holds 75 source-level repair tasks, 40 built around a verifier proof obligation and 35 minimized from open-source projects such as Cilium [3], xdp-tools [23], and bpftime [32], and each task ships the buggy program with an executable oracle. BPFix-BENCH holds the 235 reproduced rejections from §2.2, each carrying

backwards?

layer
is
back
but
not
directly
detected?

Fig 1?

do model improvements
mean better for humans?
(assumption)

a hand-assigned responsibility layer. RQ1 runs on BPFIX-TEST, and RQ2 and RQ3 run on BPFIX-BENCH.

RQ1: Repair effectiveness. We test whether more rejected programs can be repaired from the BPFix diagnostic than from the raw verifier log. We give a language model the buggy program with either the raw log or the BPFix diagnostic, and the model writes a candidate fix. An executable oracle independent of BPFix accepts a fix only if the program loads through the kernel verifier and passes the case’s functional and source-semantics tests. We score each task at one shot, to test the diagnostic on its own, and after one failure-informed retry, to test whether the help persists. Our primary model is Qwen3.6 27B, and we also run the hosted GLM 5.2 and a small Qwen2.5 3B at temperature zero. The 3B shows whether the gain depends on model capacity.

Replacing the raw log with the BPFix diagnostic raises repair under every model (Figure 4). On Qwen3.6 27B, the model repairs 22 of the 75 cases from the raw log in one shot and 38 from the diagnostic, and one retry raises the two to 30 and 44. GLM 5.2 shows the same one-shot gain, 28 against 38, though the retry narrows the gap, 47 against 52. The small Qwen2.5 3B repairs none of the 75 from the raw log, and 8 from the diagnostic in one shot and 10 with one retry. Because the gain holds under three different models, it comes from the diagnostic.

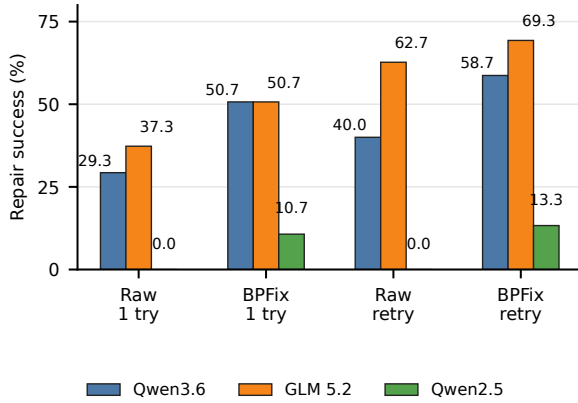


Figure 4. BPFIX-TEST repair success across three models. Bars are grouped by prompt mode, and color identifies the model. Retry bars use one failure-informed retry. BPFix improves one-shot repair for all three models and remains ahead under retry.

To locate the gain, we charge each failed candidate to the first stage it fails: returning a program, compiling, loading through the verifier, passing the functional test, or passing the source-semantics test (Table 3). The diagnostic mainly removes verifier-load and source-semantics failures: on the 27B one-shot run, load failures fall from 19 to 10 and source-semantics failures from 22 to 16, while compile failures stay

Table 3. Where repairs fail on BPFIX-TEST, by stage, for the one-attempt runs. The BPFix diagnostic mainly removes verifier-load (Load) and proof failures; compile and model-call (Call) failures stay low. Each entry counts failing cases at a stage, and pass count plus the row total is 75.

Model	Input	Compile	Load	Func.	Proof	Call
Qwen3.6 27B	raw	3	19	9	22	0
	BPFix	1	10	10	16	0
GLM 5.2	raw	1	10	11	25	0
	BPFix	1	5	9	22	0
Qwen2.5 3B	raw	7	62	0	3	3
	BPFix	14	39	6	8	0

(a) Rejected program, verifier line, and BPFix diagnostic

```
18 __u64 *raw_map = (__u64 *)&globals; // &globals: the map object pointer
19 __u64 v = *raw_map;
20 *raw_map = v + 1; // rejected here
```

verifier: only read from bpf_array is supported

```
error[BPFIX-E010]: map pointer is accessed as ordinary memory
= class: source_bug --> prog.c:20
= required proof: derive a map-value pointer with the proper
  map helper before reading or writing map contents
= next action: look up an element, then write through that pointer
```

(b) Fix that follows the diagnostic

```
__u32 key = 0;
__u64 *v = bpf_map_lookup_elem(&globals, &key);
if (!v) return 0;
*v += 1;
```

Figure 5. A real rejection from the aya project (issue 1002). The program treats the map object &globals as a value pointer and writes through it, which the verifier rejects with one terminal line. BPFix recovers the error class, the rejected span, the required proof, and a next action. The fix in (b) looks up the value and writes through the returned pointer.

low. The diagnostic therefore helps the model both satisfy the verifier and preserve the program’s behavior. The 3B’s model-call failures are a length effect: three raw prompts overflowed its context window and returned no program, while the shorter diagnostic stayed within it.

RQ2: Diagnostic completeness. The bare verifier message names only the failing operation, yet a repair needs more (§2). RQ2 measures what the BPFix diagnostic adds. We run BPFix on each of the 235 BPFIX-BENCH rejections and record, against the raw message, which of four repair fields each diagnostic supplies: a stable error id, the rejected source span, the safety proof the verifier needs, and a next action.

Every diagnostic supplies all four fields, where the raw message supplies none. Figure 5 shows one rejection where the terminal line gives only the symptom and BPFix adds all four. BPFix produces a supported diagnostic for every rejection in the corpus, across 21 error families, with no fallback, though the same corpus guided BPFix’s development.

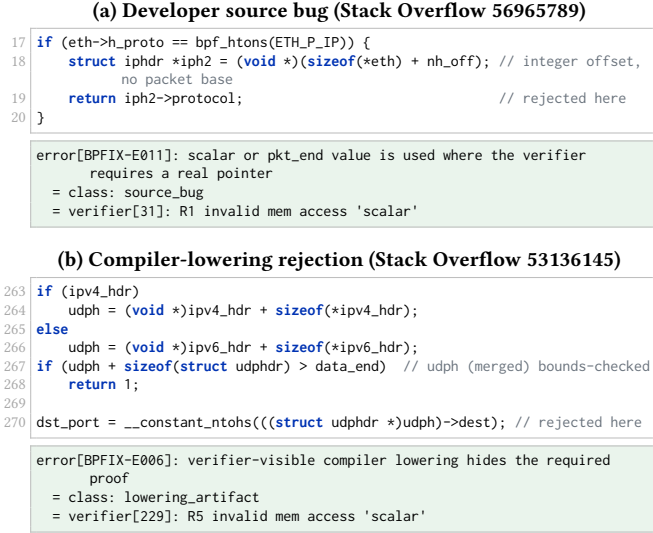


Figure 6. Two real rejections that print the same verifier message invalid mem access 'scalar' but belong to different layers. In (a) the developer casts an integer offset to a pointer with no packet base, and BPFix reports `source_bug`. In (b) the source bounds-checks the packet pointer correctly, yet the compiler's branch merge drops its packet-pointer type, and BPFix reports `lowering_artifact`. The verifier message alone cannot separate the two. Each diagnostic is abridged to its error id, class, and the verifier line it explains.

RQ3: Layer attribution. BPFix reports the responsible layer as a best-effort heuristic (§3). The BPFIX-BENCH layer labels were developed alongside BPFix, so we examine layer separation through a case study on one message that two layers share. The message invalid mem access 'scalar' prints in BPFIX-BENCH for both source bugs and compiler-lowering rejections (§2). We study one rejection of each kind, and the compiler-lowering rejection has correct source with an upstream fix that steers the compiler, so its layer holds independent of BPFix.

Figure 6 shows the two rejections. Both print invalid mem access 'scalar', yet BPFix reports `source_bug` for the integer-to-pointer cast in (a) and `lowering_artifact` for the correctly bounds-checked packet pointer in (b). The message is identical, so BPFix supplies the layer the message leaves out.

5 Related Work

Three lines of work bear on BPFix, listed from the eBPF domain outward, and each differs from it in the input it consumes or the output it produces.

eBPF verification and safety. Prior eBPF tooling decides whether a program is safe or sidesteps the verifier, and explaining why a rejected program failed is left to the developer. Empirical studies document the cost: verifier rejections are

hard to read and push developers to rewrite working code to pass the verifier [5, 7]. Offline re-provers such as PREVAIL establish safety over their own abstract domain without reading a rejection log [8], and verifier audits check the soundness of its range and tnum operators or find logic bugs that admit unsafe programs [22, 24, 25]. Others sidestep the verifier altogether, replacing it with safe Rust or program synthesis [9, 31], optimizing the bytecode before it loads [18, 28], or isolating programs at runtime [6, 13, 14, 17, 30]. None replays the verifier's own rejection log to explain why the safety fact was lost; attributing that loss to a layer remains a best-effort prototype (§3).

Error diagnosis for compilers and checkers. Tools that diagnose a rejection exist outside eBPF, but they read facts the checker built during its own check. Type-error diagnosis localizes the source responsible for an ill-typed program from the constraints or facts the type checker itself produced [11, 19, 21, 29], and surveys document why raw compiler messages are unhelpful [2]. For eBPF, Pretty Verifier maps the verifier's error to a C source line and rewords it from the source, the object file's debug information, and the verifier output [20]. These tools read facts the checker owns, or reword its message at the source level, and none reconstructs the abstract-state lifecycle a rejection log records. BPFix recovers that state, such as register types, scalar ranges, and pointer provenance [16], from the verifier log the kernel produced, below the source language where the lowering-artifact class originates [1, 15].

Fault localization and proof explanation. Techniques that pinpoint or explain a failure predate eBPF, but each needs an input BPFix does not have. Spectrum-based fault localization ranks suspect statements by how their execution correlates with test failures, needing many runs and a test suite [10, 27]; program slicing extracts the statements a value depends on, needing the program's dependence graph [26]; and unsat-core extraction names a minimal infeasible subset of constraints from a solver's structured search artifacts [4]. BPFix works from a single verifier rejection log, explaining why the program was rejected. The kernel verifier simulates execution over an abstract state and emits no such artifact [8, 16].

6 Conclusion

An eBPF program loads only after a static verifier proves it safe, and on rejection the verifier returns a message that does not say how to repair the program. Across BPFIX-BENCH, a corpus of 235 reproduced rejections, this message underdetermines the repair, leaving the root cause and the responsible layer for the developer to recover. Reading only the log, BPFix reconstructs the safety proof the verifier discarded and recovers the root cause from the verifier's own per-instruction states. On BPFIX-TEST, a 75-case repair suite, the BPFix diagnostic raises a downstream model's repair

from 22 of 75 to 38 in one shot, and from 30 to 44 with one retry. Attributing the responsible layer remains best-effort, and a frozen heldout benchmark is future work.

References

- [1] Daroc Alden. 2024. LLVM Improvements for BPF Verification. LWN.net. <https://lwn.net/Articles/974945/>, accessed 2026-06-17.
- [2] Brett A. Becker, Paul Denny, Raymond Pettit, Durell Bouchard, Dennis J. Bouvier, Brian Harrington, Amir Kamil, Amey Karkare, Chris McDonald, Peter-Michael Osera, Janice L. Pearce, and James Prather. 2019. Compiler Error Messages Considered Unhelpful: The Landscape of Text-Based Programming Error Message Research. In *Proceedings of the Working Group Reports on Innovation and Technology in Computer Science Education (Aberdeen, Scotland Uk) (ITiCSE-WGR '19)*. Association for Computing Machinery, New York, NY, USA, 177–210. doi:10.1145/3344429.3372508
- [3] Cilium Authors. 2026. Cilium: eBPF-based Networking, Observability, and Security. <https://github.com/cilium/cilium>. Open-source project, accessed 2026.
- [4] A. Cimatti, A. Griggio, and R. Sebastiani. 2011. Computing Small Unsatisfiable Cores in Satisfiability Modulo Theories. *Journal of Artificial Intelligence Research* 40 (April 2011), 701–728. doi:10.1613/jair.3196
- [5] Mugdha Deokar, Jingyang Men, Lucas Castanheira, Ayush Bhardwaj, and Theophilus A. Benson. 2024. An Empirical Study on the Challenges of eBPF Application Development. In *Proceedings of the ACM SIGCOMM 2024 Workshop on EBPF and Kernel Extensions* (Sydney, NSW, Australia) (eBPF '24). Association for Computing Machinery, New York, NY, USA, 1–8. doi:10.1145/3672197.3673429
- [6] Kumar Kartikeya Dwivedi, Rishabh Iyer, and Sanidhya Kashyap. 2024. Fast, Flexible, and Practical Kernel Extensions. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX, USA) (SOSP '24). Association for Computing Machinery, New York, NY, USA, 249–264. doi:10.1145/3694715.3695950
- [7] Bolaji Gbadamosi, Luigi Leonardi, Tobias Pulls, Toke Høiland-Jørgensen, Simone Ferlin-Reiter, Simo Sorce, and Anna Brunström. 2024. The eBPF Runtime in the Linux Kernel. arXiv:2410.00026 [cs.OS] <https://arxiv.org/abs/2410.00026>
- [8] Elazar Gershuni, Nadav Amit, Arie Gurfinkel, Nina Narodytska, Jorge A. Navas, Noam Rinetzy, Leonid Ryzhyk, and Mooly Sagiv. 2019. Simple and precise static analysis of untrusted Linux kernel extensions. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Phoenix, AZ, USA) (PLDI 2019). Association for Computing Machinery, New York, NY, USA, 1069–1084. doi:10.1145/3314221.3314590
- [9] Jinghao Jia, Ruowen Qin, Milo Craun, Egor Lukiyanov, Ayush Bansal, Minh Phan, Michael V. Le, Hubertus Franke, Hani Jamjoom, Tianyin Xu, and Dan Williams. 2025. Rex: Closing the language-verifier gap with safe and usable kernel extensions. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association, Boston, MA, 325–342. <https://www.usenix.org/conference/atc25/presentation/jia>
- [10] James A. Jones and Mary Jean Harrold. 2005. Empirical evaluation of the tarantula automatic fault-localization technique. In *Proceedings of the 20th IEEE/ACM International Conference on Automated Software Engineering* (Long Beach, CA, USA) (ASE '05). Association for Computing Machinery, New York, NY, USA, 273–282. doi:10.1145/1101908.1101949
- [11] Benjamin S. Lerner, Matthew Flower, Dan Grossman, and Craig Chambers. 2007. Searching for type-error messages. In *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation* (San Diego, California, USA) (PLDI '07). Association for Computing Machinery, New York, NY, USA, 425–434. doi:10.1145/1250734.1250783
- [12] libbpf. 2025. bpfv: BPF Verifier Visualizer. <https://github.com/libbpf/bpfv>. Accessed 2026-06-25.
- [13] Soo Yee Lim, Xueyuan Han, and Thomas Pasquier. 2023. Unleashing Unprivileged eBPF Potential with Dynamic Sandboxing. In *Proceedings of the 1st Workshop on EBPF and Kernel Extensions* (New York, NY, USA) (eBPF '23). Association for Computing Machinery, New York, NY, USA, 42–48. doi:10.1145/3609021.3609301
- [14] Soo Yee Lim, Tanya Prasad, Xueyuan Han, and Thomas Pasquier. 2024. SafeBPF: Hardware-assisted Defense-in-depth for eBPF Kernel Extensions. In *Proceedings of the 2024 on Cloud Computing Security Workshop* (Salt Lake City, UT, USA) (CCSW '24). Association for Computing Machinery, New York, NY, USA, 80–94. doi:10.1145/3689938.3694781
- [15] Linux kernel documentation. 2024. BPF LLVM Relocations. https://docs.kernel.org/bpf/llvm_reloc.html. The Linux Kernel documentation, BPF subsystem. Accessed 2026-06-15.
- [16] Linux kernel documentation. 2025. eBPF verifier. <https://docs.kernel.org/bpf/verifier.html>. The Linux Kernel documentation, Documentation/bpf/verifier.rst.
- [17] Hongyi Lu, Shuai Wang, Yechang Wu, Wanning He, and Fengwei Zhang. 2024. MOAT: Towards Safe BPF Kernel Extension. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 1153–1170. <https://www.usenix.org/conference/usenixsecurity24/presentation/lu-hongyi>
- [18] Jinsong Mao, Hailun Ding, Juan Zhai, and Shiqing Ma. 2024. Merlin: Multi-tier Optimization of eBPF Code for Performance and Compactness. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) (ASPLOS '24). Association for Computing Machinery, New York, NY, USA, 639–653. doi:10.1145/3620666.3651387
- [19] Zvonimir Pavlinovic, Tim King, and Thomas Wies. 2014. Finding minimum type error sources. *SIGPLAN Not.* 49, 10 (Oct. 2014), 525–542. doi:10.1145/2714064.2660230
- [20] Rosario Rizza, Riccardo Sisto, and Fulvio Valenza. 2025. Design and implementation of a tool to improve error reporting for eBPF code. In *2025 IEEE International Conference on Cyber Security and Resilience (CSR)*. 214–219. doi:10.1109/CSR64739.2025.11130075
- [21] Eric L. Seidel, Huma Sibghat, Kamalika Chaudhuri, Westley Weimer, and Ranjit Jhala. 2017. Learning to blame: localizing novice type errors with data-driven diagnosis. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 60 (Oct. 2017), 27 pages. doi:10.1145/3138818
- [22] Hao Sun and Zhendong Su. 2024. Validating the eBPF verifier via state embedding. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) (OSDI'24). USENIX Association, USA, Article 33, 14 pages.
- [23] The XDP Project. 2026. xdp-tools: Utilities and Example Programs for Use with XDP. <https://github.com/xdp-project/xdp-tools>. Open-source project, accessed 2026.
- [24] Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, and Santosh Nagarakatte. 2022. Sound, precise, and fast abstract interpretation with tristate numbers. In *Proceedings of the 20th IEEE/ACM International Symposium on Code Generation and Optimization* (Virtual Event, Republic of Korea) (CGO '22). IEEE Press, 254–265. doi:10.1109/CGO53902.2022.9741267
- [25] Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, and Santosh Nagarakatte. 2023. Verifying the Verifier: eBPF Range Analysis Verification. In *Computer Aided Verification: 35th International Conference, CAV 2023, Paris, France, July 17–22, 2023, Proceedings, Part III* (Paris, France). Springer-Verlag, Berlin, Heidelberg, 226–251. doi:10.1007/978-3-031-37709-9_12
- [26] Mark Weiser. 1981. Program slicing. In *Proceedings of the 5th International Conference on Software Engineering* (San Diego, California, USA) (ICSE '81). IEEE Press, 439–449.
- [27] W. Eric Wong, Ruizhi Gao, Yihao Li, Rui Abreu, and Franz Wotawa. 2016. A Survey on Software Fault Localization. *IEEE Transactions on Software Engineering* 42, 08 (Aug. 2016), 707–740. doi:10.1109/TSE.2016.2521368

- [28] Qiongwen Xu, Michael D. Wong, Tanvi Wagle, Srinivas Narayana, and Anirudh Sivaraman. 2021. Synthesizing safe and efficient kernel extensions for packet processing. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference (Virtual Event, USA) (SIGCOMM '21)*. Association for Computing Machinery, New York, NY, USA, 50–64. doi:10.1145/3452296.3472929
- [29] Danfeng Zhang and Andrew C. Myers. 2014. Toward general diagnosis of static errors. *SIGPLAN Not.* 49, 1 (Jan. 2014), 569–581. doi:10.1145/2578855.2535870
- [30] Peihua Zhang, Chenggang Wu, Xiangyu Meng, Yinqian Zhang, Mingfan Peng, Shiyang Zhang, Bing Hu, Mengyao Xie, Yuanming Lai, Yan Kang, and Zhe Wang. 2024. HIVE: A Hardware-assisted Isolated Execution Environment for eBPF on AArch64. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 163–180. <https://www.usenix.org/conference/usenixsecurity24/presentation/zhang-peihua>
- [31] Yusheng Zheng, Yiwei Yang, Maolin Chen, and Andrew Quinn. 2024. Kgent: Kernel Extensions Large Language Model Agent. In *Proceedings of the ACM SIGCOMM 2024 Workshop on EBPF and Kernel Extensions (Sydney, NSW, Australia) (eBPF '24)*. Association for Computing Machinery, New York, NY, USA, 30–36. doi:10.1145/3672197.3673434
- [32] Yusheng Zheng, Tong Yu, Yiwei Yang, Yanpeng Hu, Xiaozheng Lai, Dan Williams, and Andi Quinn. 2025. Extending applications safely and efficiently (*OSDI '25*). USENIX Association, USA, Article 31, 18 pages.